

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Use Case :

WhatsNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.

The implementation of this Salesforce project at WhatsNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.

Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall

productivity of the company but also allows for a more agile response to market demands and customer needs.

Requirements :

1. Salesforce CRM Implementation

- Store and manage vehicle details, stock availability, and dealer information in Salesforce.
- Track customer orders, test drives, and service requests efficiently.
- Automate workflows to assign orders to the nearest dealer based on customer location.

2. Process Automation

- Prevent order placement if the vehicle is out of stock.
- Auto-assign orders to the nearest dealer based on the customer's location.
- Send automated email reminders for scheduled test drives.

3. Apex and Trigger

- Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
- Use trigger handlers to follow best practices and ensure modularity and maintainability.

4. Batch Jobs

- Develop a batch Apex job to periodically check vehicle stock levels and update availability.
- Send scheduled email notifications for stock replenishment and order processing.

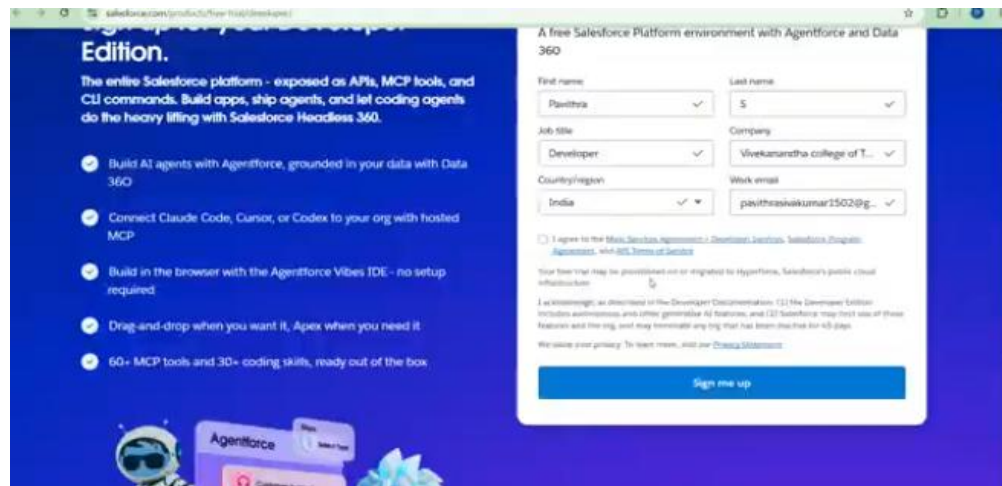
What you'll learn :

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

MileStone 1 :

Activity 1: Creating Developer Account

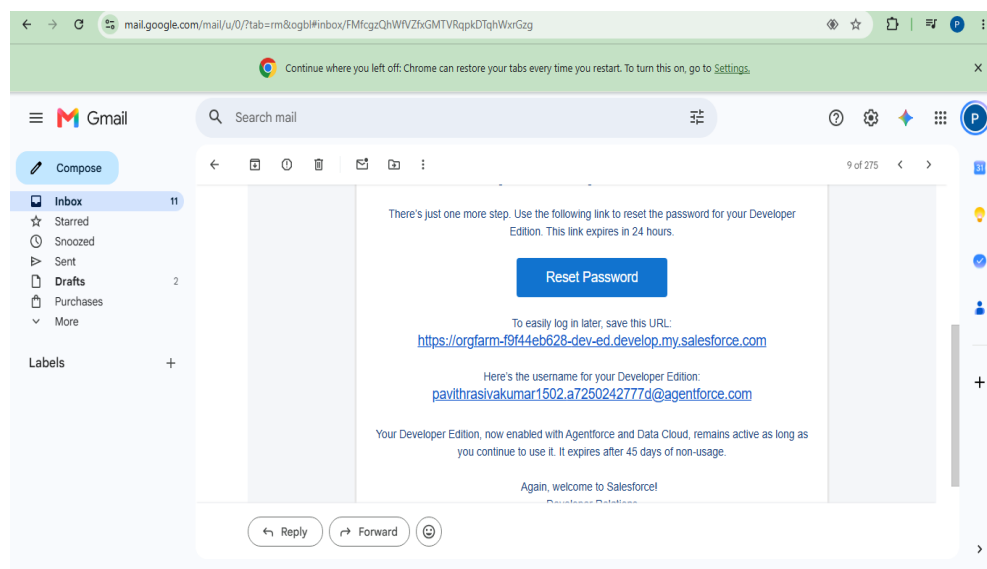
I completed the activity by creating a Salesforce Developer Account. I entered my name, email, developer role, college name, country, postal code, and username, and successfully submitted the signup form.



The screenshot shows the Salesforce Developer Edition signup page. On the left, there's a blue sidebar with the text "Edition. The entire Salesforce platform - exposed as APIs, MCP tools, and CLI commands. Build apps, ship agents, and let coding agents do the heavy lifting with Salesforce Headless 360." Below this are five bullet points: "Build AI agents with Agentforce, grounded in your data with Data 360", "Connect Claude Code, Cursor, or Codex to your org with hosted MCP", "Build in the browser with the Agentforce Vibes IDE - no setup required.", "Drag-and-drop when you want it, Apex when you need it", and "60+ MCP tools and 30+ coding skills, ready out of the box". On the right, there's a white form titled "A free Salesforce Platform environment with Agentforce and Data 360". The form has fields for "First name" (Pavithra), "Last name" (S), "Job title" (Developer), "Company" (Vivekanantha college of T...), "Country/region" (India), and "Work email" (pavithrasivakumar2502@g...). There's a checkbox for "I agree to the Salesforce Agreement / Developer Edition, Salesforce Proprietary Agreement, and API Terms of Service". Below the checkbox, there's a small text block: "Your free trial may be provisioned on or migrated to Hyperforce, Salesforce's public cloud infrastructure. I acknowledge, as described in the Developer Documentation, (1) the Developer Edition includes autonomous and other generative AI features, and (2) Salesforce may test use of these features and the org, and may monetize any org that has been checked for 10 days. We value your privacy. To learn more, visit our Privacy Statement". At the bottom of the form is a blue "Sign me up" button.

Activity 2: Account Activation

I completed the account activation by verifying my email through the Salesforce verification link. After that, I created my password and security question and successfully accessed the Salesforce Setup page.



The screenshot shows a Gmail interface with an email from Salesforce. The email body says: "There's just one more step. Use the following link to reset the password for your Developer Edition. This link expires in 24 hours." Below this is a blue "Reset Password" button. Then it says: "To easily log in later, save this URL: https://orgfarm-f9f44eb628-dev-ed.develop.my.salesforce.com". Below that, it says: "Here's the username for your Developer Edition: pavithrasivakumar1502.a7250242777d@agentforce.com". At the bottom, it says: "Your Developer Edition, now enabled with Agentforce and Data Cloud, remains active as long as you continue to use it. It expires after 45 days of non-usage. Again, welcome to Salesforce!". The email is shown in a Gmail window with a search bar, a left sidebar with "Compose", "Inbox", "Starred", "Snoozed", "Sent", "Drafts", "Purchases", and "More", and a right sidebar with "9 of 275" and a "P" icon. At the bottom of the email are "Reply" and "Forward" buttons.

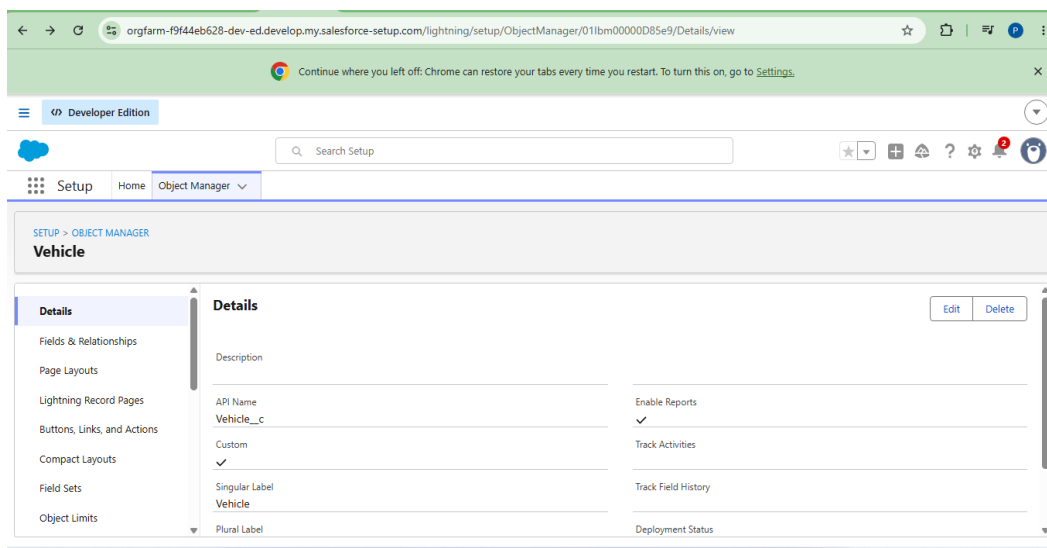
MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

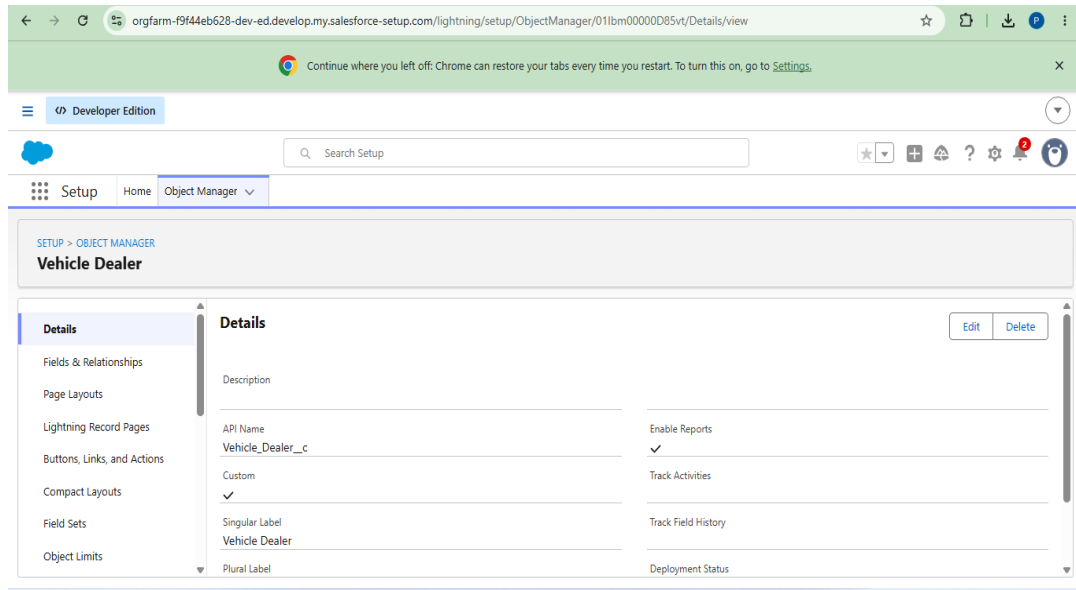
Activity 1: Create Vehicle Object

I created the Vehicle custom object to store vehicle details. I configured the Vehicle Name as the record name, enabled reporting and searching, and saved the object successfully.



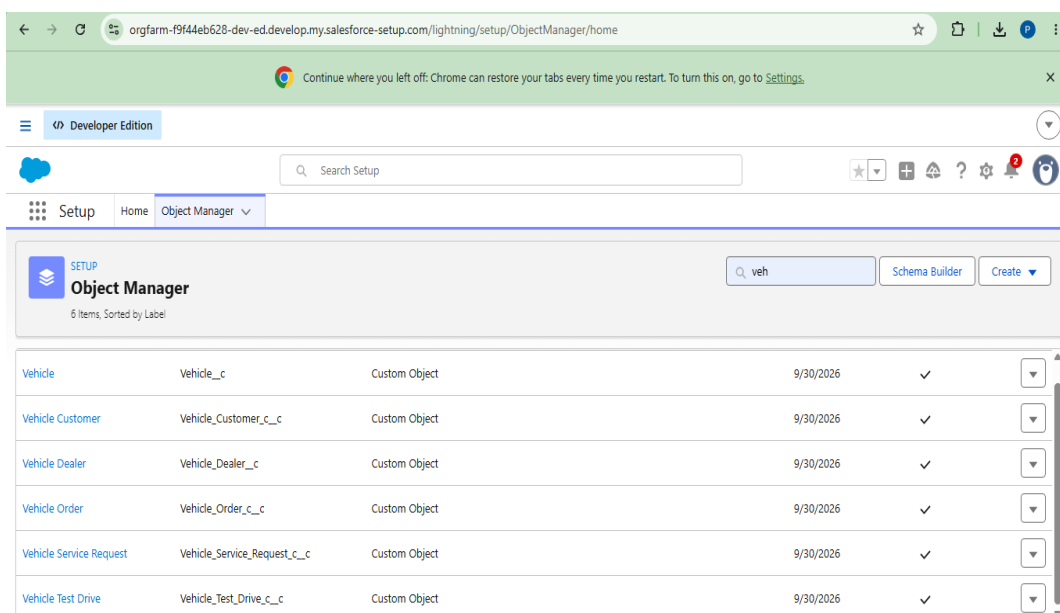
Activity 2: Create Dealer Object

I created the Vehicle Dealer custom object to maintain dealer information. I configured the Dealer Name as the record name, enabled reporting and searching, and saved the object.



Activity 3: Creating remaining objects

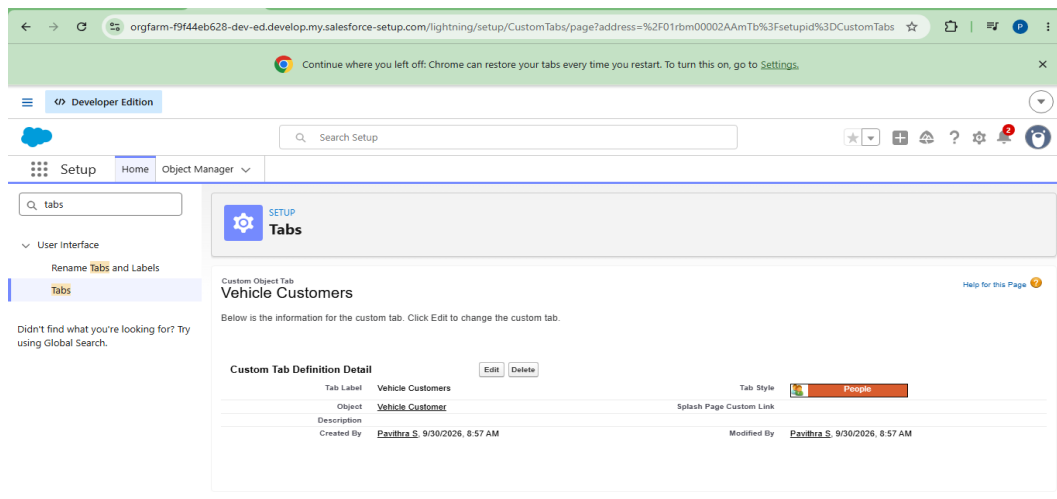
I completed the remaining custom objects required for the project, including Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Vehicle Service Request. These objects are used to manage customer, order, test-drive, and service-related information.



Milestone 3 - Tabs

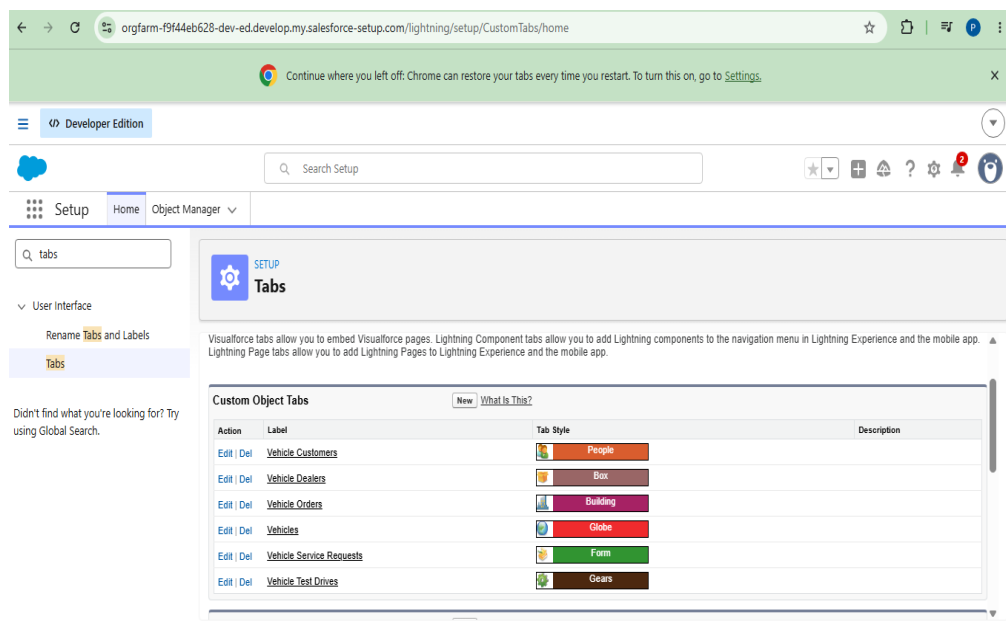
Activity 1: Creating a Custom Tab(Vehicle)

I created a custom tab for the Vehicle object from the Salesforce Setup page. I selected the Vehicle object, configured the tab style, and saved the tab successfully.



Activity 2:

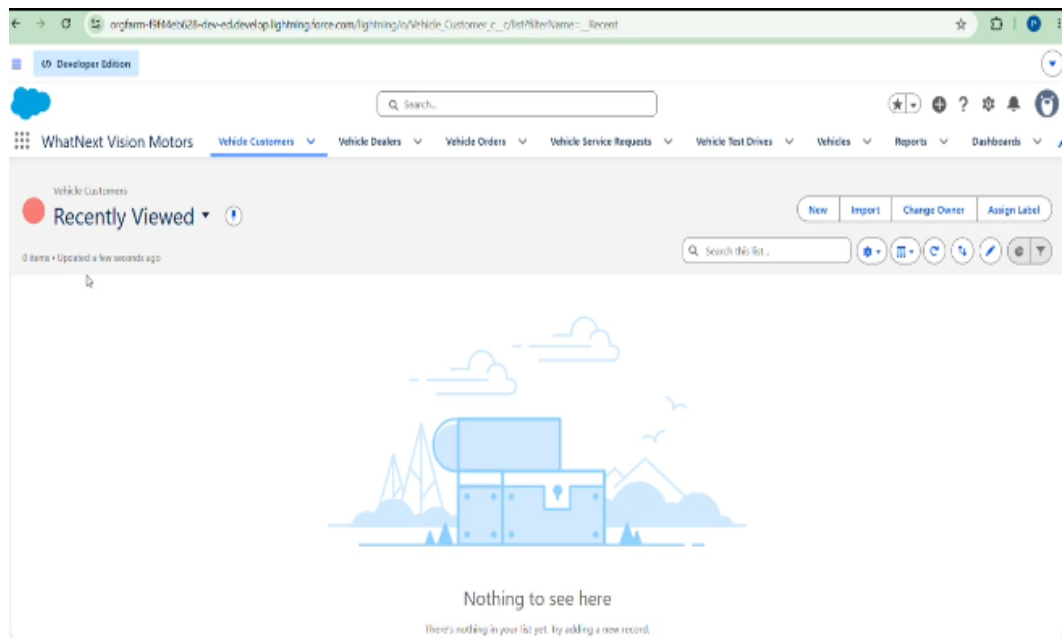
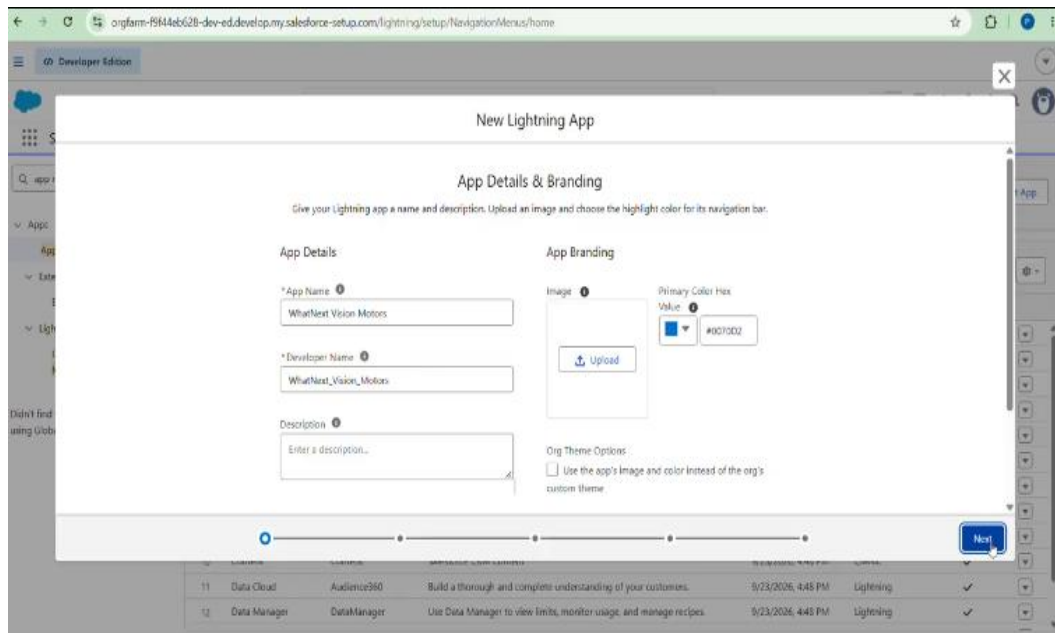
I created custom tabs for the other objects that were created in the previous milestone. This makes the objects easily accessible from the Salesforce application.



Milestone 4- The Lightning App:

Activity 1: Create a Lightning App

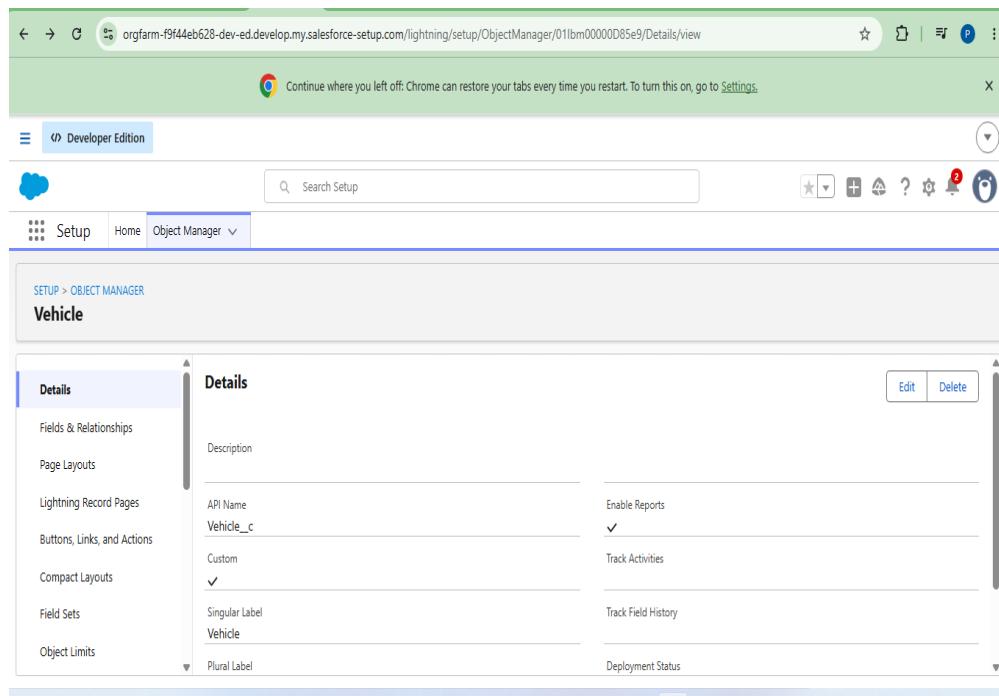
I created the WhatNext Vision Motors Lightning App. I configured the app details and added navigation items such as Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, and Dashboard, and finally added the System Administrator profile.



Milestone 5 : Fields & Relationships

Activity 1: Creating a Field in Vehicle Object

I created a Vehicle Model picklist field in the Vehicle object. I added the required picklist values and saved the field successfully.

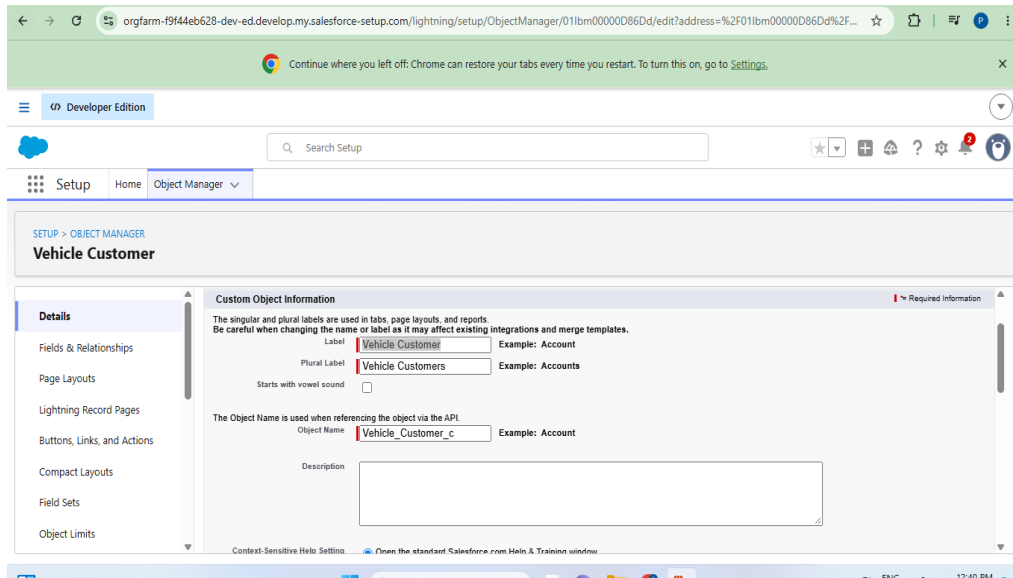


Activity 2 : Creating Stock Quantity field in vehicle object

I created the Stock Quantity field in the Vehicle object using the Number data type. This field is used to store the available quantity of each vehicle.

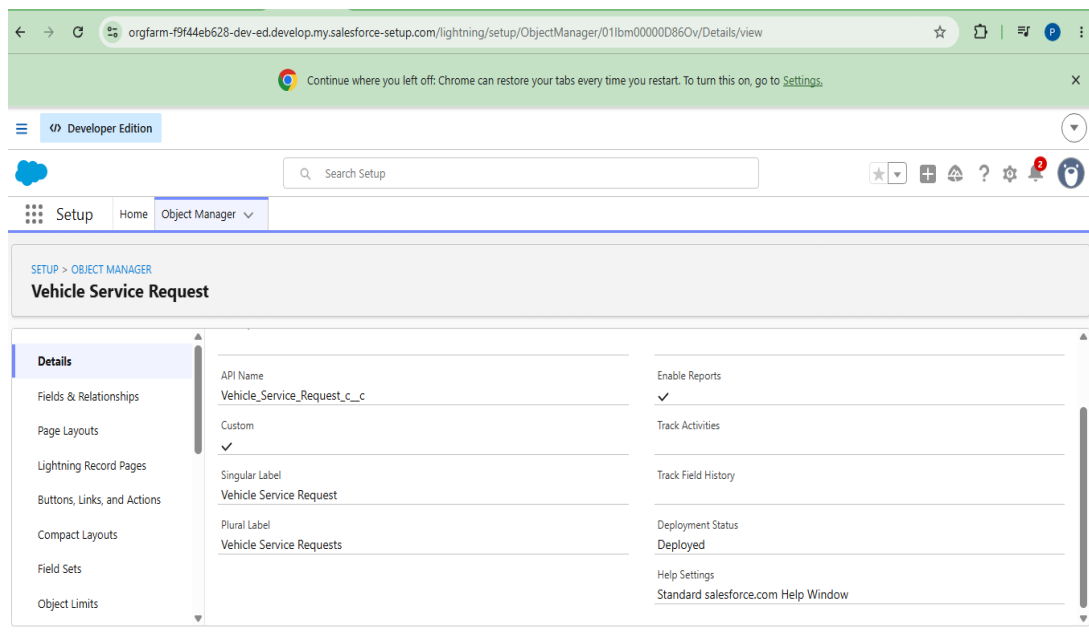
Activity 3 : Creating a Lookup Relationship in Vehicle Object to Dealer Object

I created a Lookup Relationship between the Vehicle and Dealer objects. This relationship connects each vehicle with the corresponding dealer information.



Activity 4 : Creating Remaining Objects and Fields.

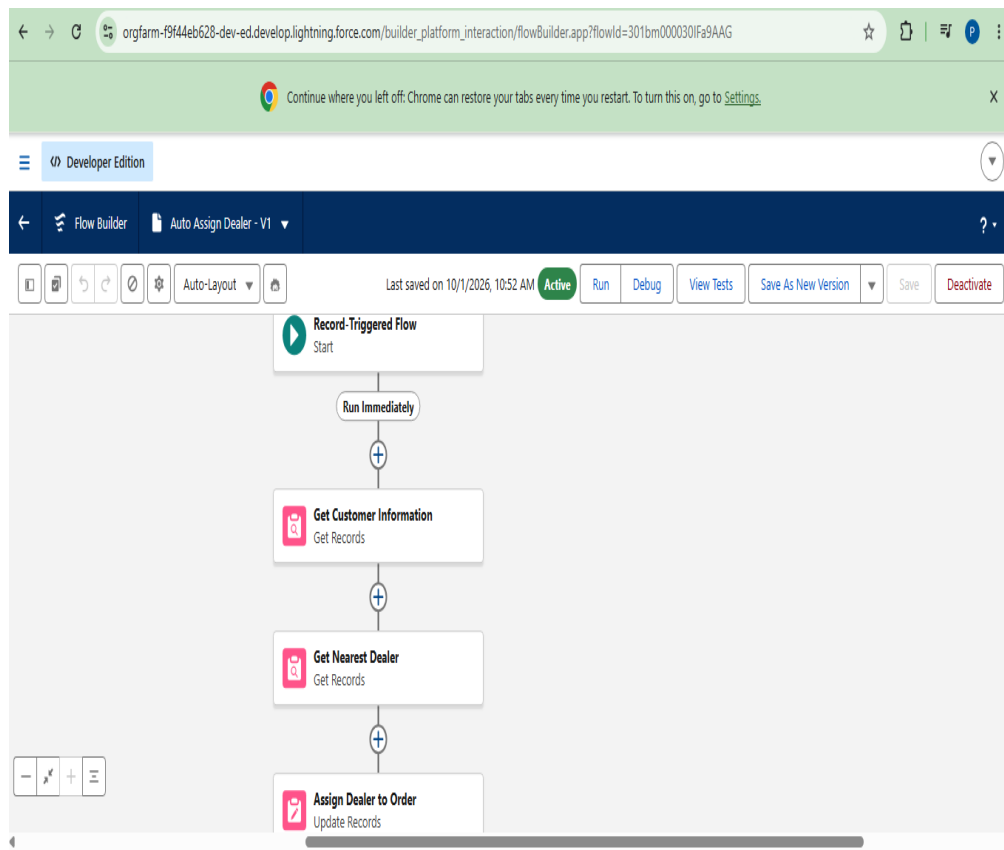
I completed the remaining required objects and fields for the project. These fields support vehicle, dealer, customer, order, test-drive, and service-request information.



MileStone 6 :

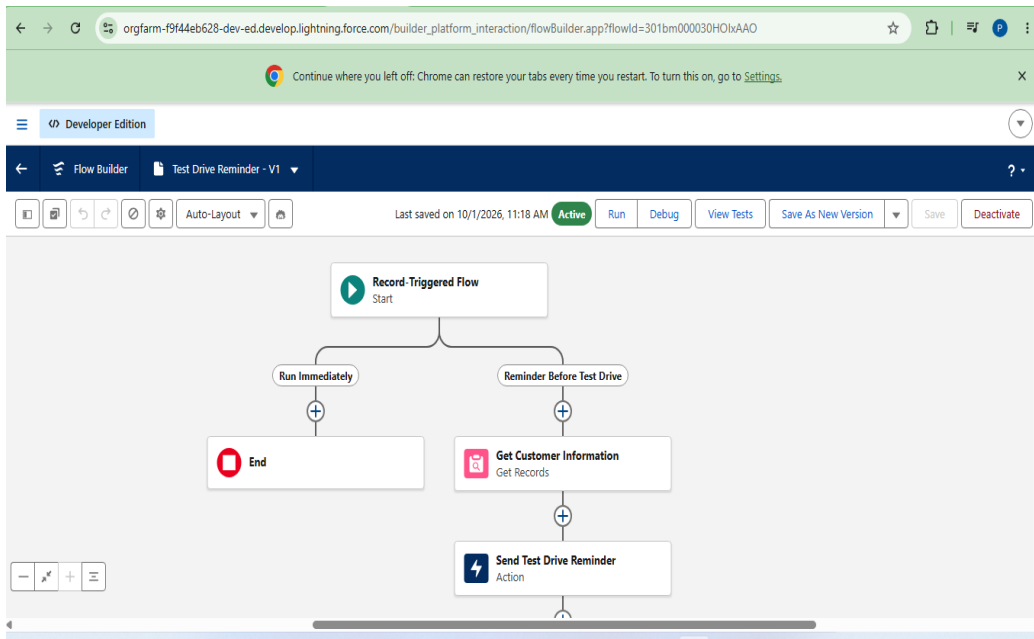
Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

I created a record-triggered flow for the Vehicle Order object. When an order is created with Pending status, the flow gets the customer information, finds the dealer based on the customer's location, and assigns the dealer to the order.



Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

I created another record-triggered flow for the Vehicle Test Drive object. When a test drive is scheduled, the flow creates a scheduled path and sends an email reminder to the customer one day before the test drive.



MileStone 7 :

Create Apex and Trigger Batch Jobs :

Activity 1 – Create Apex Class

I created the VehicleOrderTriggerHandler Apex class in the Developer Console. The class contains the business logic for checking vehicle stock and updating stock when an order is confirmed.

```
orgfarm-19f44eb628-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage
File Edit Debug Test Workspace Help < >
VehicleOrderTriggerHandler.apex VehicleOrderTrigger.apxt VehicleOrderBatch.apxc VehicleOrderBatchScheduler.apxc
Code Coverage: None API Version: 67 Go To
139 if (order.Status__c == 'Confirmed') {
140     shouldUpdate = true;
141 }
142
143 } else {
144
145     Vehicle_Order_c_c oldOrder =
146         oldOrders.get(order.Id);
147
148     if (oldOrder != null &&
149         order.Status__c == 'Confirmed' &&
150         oldOrder.Status__c != 'Confirmed') {
151
152         shouldUpdate = true;
153     }
154 }
```

Activity 2 – Prevent Out-of-Stock Orders

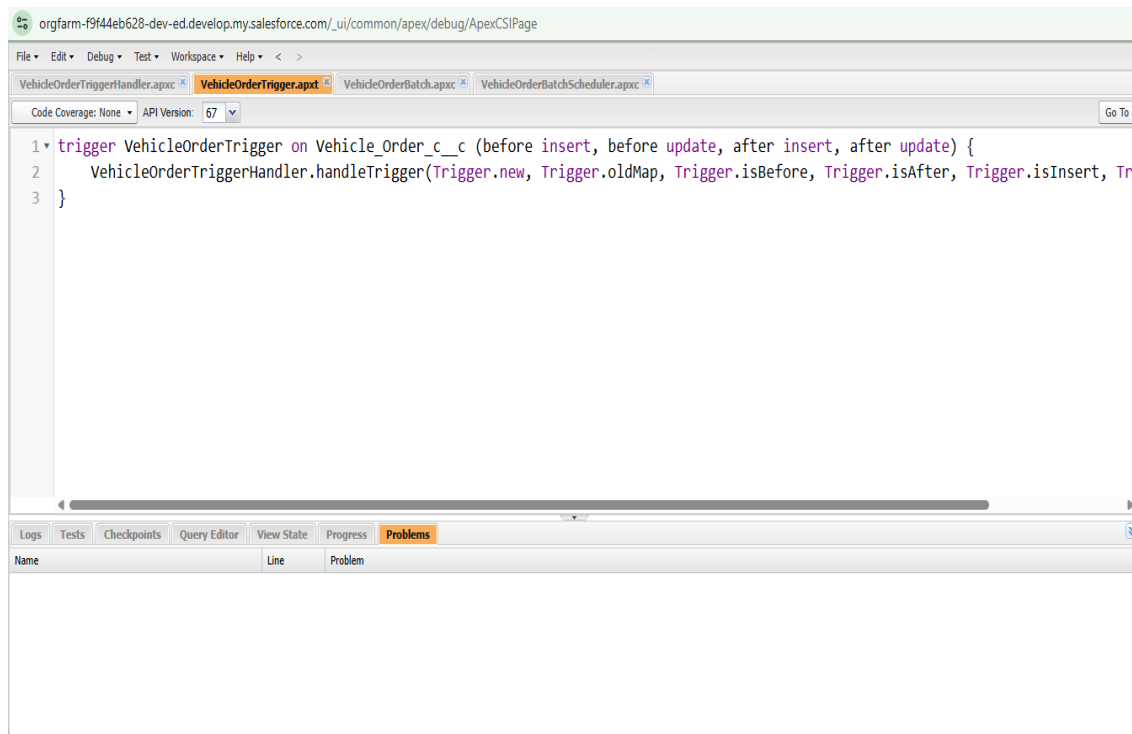
I implemented the Apex logic to check the vehicle's stock quantity before placing an order. If the stock quantity is zero or less, the system displays an error and prevents the order from being placed.

Activity 3 – Update Vehicle Stock

is then saved I implemented the logic to automatically reduce the vehicle stock when a confirmed order is placed. The updated stock quantity back to the Vehicle record.

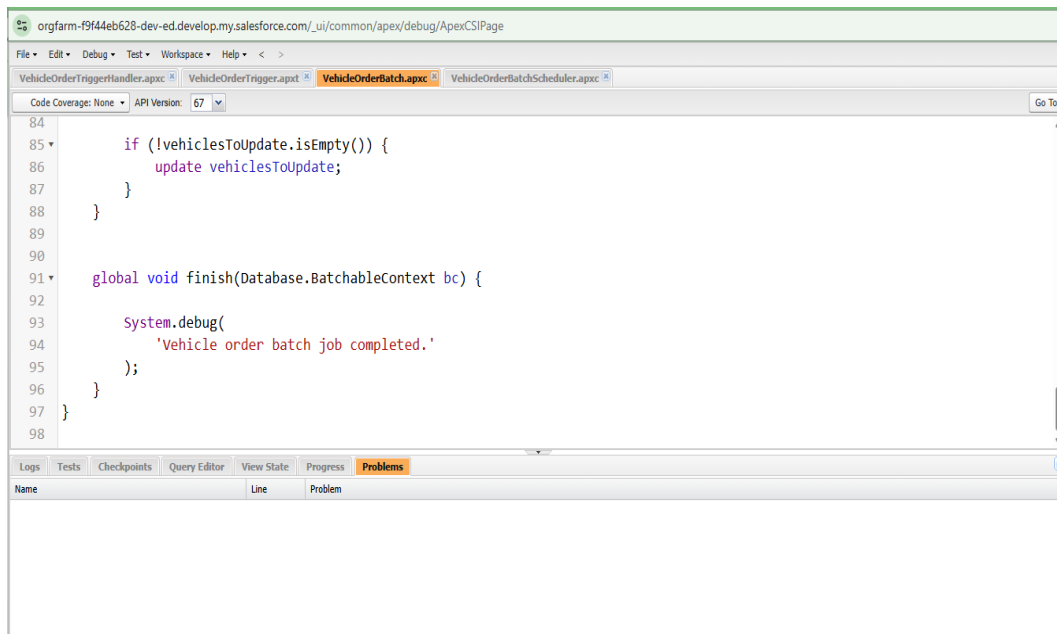
Activity 4 – Create Trigger

I created the Vehicle Order trigger and connected it with the Apex trigger handler class. The trigger calls the required Apex logic during order creation or update.



Activity 5 – Create Batch Job

I created the Batch Apex process for vehicle orders. When a vehicle is out of stock, the order remains pending, and after stock is available, the batch process updates the order accordingly.



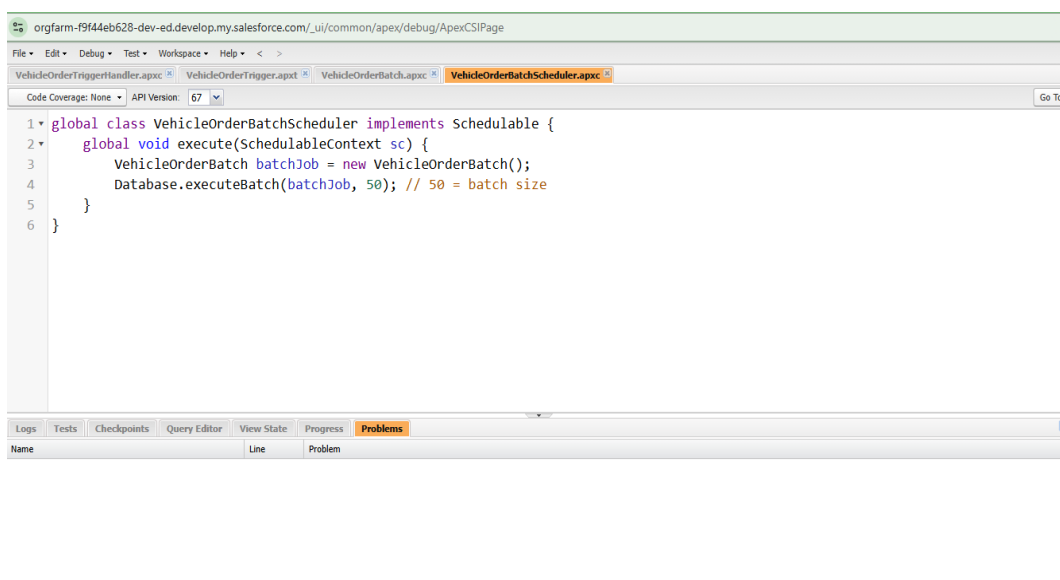
The screenshot shows the Salesforce IDE interface with the 'VehicleOrderBatch.apex' file open. The code is as follows:

```
84
85     if (!vehiclesToUpdate.isEmpty()) {
86         update vehiclesToUpdate;
87     }
88 }
89
90
91 global void finish(Database.BatchableContext bc) {
92
93     System.debug(
94         'Vehicle order batch job completed.'
95     );
96 }
97 }
98
```

The IDE interface includes a menu bar (File, Edit, Debug, Test, Workspace, Help), a toolbar with 'Code Coverage: None' and 'API Version: 67', and a 'Go To' button. The bottom pane shows tabs for 'Logs', 'Tests', 'Checkpoints', 'Query Editor', 'View State', 'Progress', and 'Problems'.

Activity 6 – Create Schedule Class

I created the VehicleOrderBatchScheduler class to execute the batch job automatically. The scheduler calls the Vehicle Order Batch class with a batch size of 50.



The screenshot shows the Salesforce IDE interface with the 'VehicleOrderBatchScheduler.apex' file open. The code is as follows:

```
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50); // 50 = batch size
5     }
6 }
```

The IDE interface includes a menu bar (File, Edit, Debug, Test, Workspace, Help), a toolbar with 'Code Coverage: None' and 'API Version: 67', and a 'Go To' button. The bottom pane shows tabs for 'Logs', 'Tests', 'Checkpoints', 'Query Editor', 'View State', 'Progress', and 'Problems'.

Activity 7 – Schedule the Batch Job

I scheduled the batch job to run automatically using a CRON expression. This allows the Vehicle Order processing to run on a scheduled basis without manual execution.

```
Enter Apex Code
1 String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
2 System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
3
```

Open Log Execute Execute Highlighted

orgfarm-f9f44eb628-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ScheduledJobs/home

Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to [Settings](#).

Developer Edition

Search Setup

Setup Home Object Manager

job

Environments

Jobs

Apex Flex Queue

Apex Jobs

Background Jobs

Bulk Data Load Jobs

Scheduled Jobs

Didn't find what you're looking for? Try using Global Search.

SETUP Scheduled Jobs

You have currently used 1 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: All Scheduled Jobs Create New View

Schedule Apex							
Action	Job Name ↑	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Vehicle Order Processing	S. Pavithra	9/30/2026, 11:43 PM		10/1/2026, 12:00 PM	Scheduled Apex	08ebm00000yv0KS
Del	Metalytics Data Loader Job for Org : 00D0m000011TqGf	User Integration	9/23/2026, 4:53 PM	9/30/2026, 6:26 PM	10/1/2026, 6:26 PM	Autonomous Data Loader Job	08ebm00000ximOz
	Program Milestone Computation Cron Job	Process, Automated	9/23/2026, 4:53 PM	9/30/2026, 5:00 PM	9/30/2026, 11:59 PM	Program Milestone Computation Cron Job	08ebm00000ximOx
	Program Status Update Cron Job	Process, Automated	9/23/2026, 4:53 PM	9/30/2026, 8:01 PM	10/1/2026, 5:00 AM	Program Status Update Cron Job	08ebm00000ximOy

Thank you